

Databases II

Semester 4 (Theory)

Mathis Neunzig

DHBW Mosbach

Autumn 2026

Table of Contents

- 1 Introduction
- 2 Data Warehouse
- 3 DWH Architectures
- 4 Creating a Data Warehouse
- 5 OLAP & Business Intelligence

Table of Contents

- 1 Introduction
- 2 Data Warehouse
- 3 DWH Architectures
- 4 Creating a Data Warehouse
- 5 OLAP & Business Intelligence

About the Lecturer

Mathis Neunzig

- B.Sc. Business Information Technology (DHBW Mannheim)
- M.Sc. Data Science (Fernuniversität Hagen)
- ABAP Developer & Trainer at SAP
- Teaches Databases & Software Engineering at DHBW
- Based in Mannheim (Baden-Württemberg)
- Originally from Oldenburg (Lower Saxony)

Contact

Email

mathis.neunzig@gmail.com

Phone

+49 174 9885992

Getting to Know You

Let's go around the room – please tell us:

- 1 Your **name**
- 2 Your **employer**
- 3 Your **favourite animal**
- 4 Your **favourite song, album or artist** right now
- 5 Your **favourite programming language** (so far)

Agenda

This course is structured around multiple interconnected topic groups that build on each other progressively throughout the semester.

1. Foundations

- Databases and Database Management Systems (DBMS)
- Data models
- Relational algebra
- Relational databases
- Data modelling & the DB design process

2. SQL

- Data Manipulation Language (DML)
- Data Definition Language (DDL)
- Data Control Language (DCL)

3. Advanced Topics

- Database transaction management
- Concurrency control
- Performance optimisation
- Backup & Recovery
- Security & SQL Injection
- Object-Relational Mapping (ORM)

4. DB Project (4th semester)

- Recap of DB design process
- Recap of ORM
- Case study: designing a database for a real-world scenario

Semester Schedule

Session	Date	Time	Topics
1	09.09	12:30–15:00	Introduction; Course overview; Exam info; DB Management System; Data Models
2	15.09	14:00–17:15	Relational Databases; Basic Relational Algebra
3	19.09	09:00–12:15	Advanced Relational Algebra; Normal Forms (1NF, 2NF, 3NF)
4	26.09	09:00–12:15	Normal Forms (BCNF, 4NF, 5NF, 6NF)
5	29.09	14:00–17:15	ERM Design
6	13.10	14:00–17:15	SQL (DML)
7	20.10	14:00–17:15	SQL (DML)
8	27.10	14:00–17:15	SQL (DCL); CAP-Theorem
9	03.11	14:00–17:15	Backup & Recovery; Performance; Concurrency
10	10.11	14:00–17:15	Security; Object-Relational Mapping
11	17.11	14:00–17:15	Exam preparation
12	26.11	10:00–11:00	Written Exam

Mailing List & Contact

- All course announcements will be sent via discord.
- There is no mailing list in place for this class. Please check the discord channel regularly for updates.
- Lecturer contact: `mathis.neunzig@gmail.com` +49 174 9885992

Emergency Contact

If you have an urgent issue that cannot wait for the next session – for example a technical problem on the day of an assignment submission – please use the phone number above. For non-urgent matters, email or discord is always preferred.

General Conduct

Standard DHBW rules of conduct apply throughout all sessions.

- **Absences:** Please give advance notice if you cannot attend. A quick message the day before is always appreciated. If you miss a session, it is your responsibility to catch up on the material.
- **Deadlines:** If you foresee that you cannot meet an assignment deadline, contact the lecturer *before* the deadline, not after. Extensions can often be arranged if asked in advance.
- **Open door:** Feel free to reach out at any time with questions, concerns, or feedback; about the content, the pace, or anything else. There are no silly questions in this course.
- **Group exercises:** Active participation during group exercises is expected and contributes to your own learning as well as the group's. Sitting passively during a group task is not acceptable.

Assessment: Written Exam

Key Facts

- **Weight:** 40 points (40 % of final grade)
- **Timing:** End of 3rd semester (12, 26.11)
- **Duration:** 60 minutes

Exam Scope

Topics marked with (!) in the slides will *not* appear in the written exam. These are advanced or supplementary topics included for completeness and professional context, but you will not be tested on them under exam conditions.

Examples

Permitted Aids You may bring **one handwritten A4 sheet** (both sides allowed) as a personal cheat sheet. It must be handwritten – printed or typed sheets are not permitted. Writing your own cheat sheet is itself an effective revision strategy.

Assessment: Assignments and Project

Overview – 60 points total (60 % of final grade)

Assignment	Description	Points
<i>Semester 3</i>		
Assignment 1	Individual work	10 pts
Assignment 2	Individual work	10 pts
<i>Semester 4</i>		
Project	Final project (in groups)	40 pts

Submission Policy (Assignments 1 and 2)

- **1st submission** receives a grade *and* written feedback.
- **Optional 2nd submission** is possible after reviewing the feedback.
- The 2nd submission grade replaces the 1st – use this opportunity wisely.

No textbook purchase is required to pass this course.

All essential material is covered in the lecture slides and exercises.

Recommended reading (for those who want to go deeper):

- Kemper & Eickler – *Datenbanksysteme*
The German-language standard reference for database courses at German universities.
Thorough and mathematically precise.
- Garcia-Molina, Ullman & Widom – *Database Systems: The Complete Book*, 2nd ed.
Comprehensive English-language textbook covering theory and practice in depth.
- Foster & Godbole – *Database Systems: A Pragmatic Approach*, 2nd ed.
A more application-oriented introduction, well-suited as a companion to this course.
- Gumm & Sommer – *Einführung in die Informatik*, 10th ed.
Broader CS introduction, useful for contextualising databases within the wider field.

Table of Contents

- 1 Introduction
- 2 Data Warehouse**
- 3 DWH Architectures
- 4 Creating a Data Warehouse
- 5 OLAP & Business Intelligence

Motivation: Why Not Just Use a Regular Database?

Operational Databases (OLTP)

- Designed for **day-to-day transactions**: inserts, updates, deletes
- Many concurrent users, short queries, row-oriented storage
- Example: booking a flight, processing a payment, updating stock

The Problem for Analytics

- Analytical queries scan **millions of rows** – too slow on OLTP systems
- Data is scattered across **multiple systems** (ERP, CRM, ...)
- Historical data is often not kept – or not in a queryable form
- Running reports on the live DB risks **performance degradation**

What Is a Data Warehouse?

Definition (W.H. Inmon, 1992)

A Data Warehouse is a **subject-oriented**, **integrated**, **time-variant**, and **non-volatile** collection of data in support of management's decision-making process.

- Central repository that consolidates data from **multiple sources**
- Optimised for **reading** and complex analytical queries
- Stores data **historically** – typically years of history
- Foundation for Business Intelligence and reporting tools

The Four Characteristics of a DWH (Inmon)

Subject-Oriented

Organised around key business subjects (Sales, Customers, Products), not around applications or processes.

Integrated

Data from heterogeneous sources is cleaned, transformed, and stored in a **consistent format** (unified naming, units, encoding).

Time-Variant

Every record is associated with a **time stamp or period**. Data is never simply overwritten – changes are captured over time.

Non-Volatile

Data is **loaded** into the DWH and **read**, but never updated or deleted in the operational sense. It is stable and persistent.

OLTP vs. OLAP at a Glance

Property	OLTP	OLAP / DWH
Purpose	Transactions / Operations	Analysis / Reporting
Users	Clerks, applications	Analysts, managers
Query type	Simple, predefined	Complex, ad-hoc
Data scope	Current state	Historical (years)
Data volume	GB range	TB – PB range
Operations	INSERT / UPDATE / DELETE	Mainly SELECT
Schema	Highly normalised (3NF)	Denormalised (star/snowflake)
Response time	Milliseconds	Seconds – minutes
Concurrency	Thousands of users	Dozens of analysts

OLTP: A Closer Look

Online Transaction Processing (OLTP)

- Manages the **operational data** of a business in real time
- ACID properties are critical: Atomicity, Consistency, Isolation, Durability
- High write throughput; short, targeted transactions

Examples

Typical OLTP Systems

- Enterprise Resource Planning (ERP): SAP, Oracle EBS
- Customer Relationship Management (CRM): Salesforce
- Point-of-Sale systems, banking core systems

Key Takeaway

OLTP systems are the **source** of data for the DWH. They are **not** designed for analytical workloads.

Key Concepts: Dimensions and Facts

Fact Table

- Contains **measurable, quantitative data** (metrics/facts)
- Examples: revenue, quantity sold, number of page views
- Typically very large (millions to billions of rows)

Dimension Tables

- Contain **descriptive attributes** used to filter and group facts
- Examples: Customer, Product, Date, Region, Store
- Relatively small; provide context for measurements

Examples

Example: Online Shop A sale is the *fact*. The *dimensions* are: **What** was sold (Product), **Who** bought it (Customer), **When** (Date), **Where** (Region).

The Star Schema

Structure

- One central **fact table** linked to multiple **dimension tables**
- Dimension tables are **denormalised** (flat, wide tables)
- Simple joins – optimised for query performance

Picture: Star Schema Diagram

Add a diagram here showing a central FACT_SALES table connected via foreign keys to DIM_DATE, DIM_CUSTOMER, DIM_PRODUCT, and DIM_REGION. Highlight the star shape formed by the relationships.

The Snowflake Schema

Structure

- Extension of the star schema: dimension tables are **normalised**
- Dimension attributes split into sub-dimension tables
- Reduces data redundancy at the cost of more joins

Star vs. Snowflake

	Star	Snowflake
Normalisation	Denormalised	Partially normalised
Query speed	Faster	Slightly slower
Storage	More redundancy	Less redundancy
Complexity	Simple	More complex

The Date Dimension

The **date/time dimension** is present in virtually every DWH.

Examples

Typical Attributes of DIM_DATE

Column	Example value
date_key	20240315
full_date	2024-03-15
day_of_week	Friday
week_of_year	11
month_name	March
quarter	Q1
year	2024
is_holiday	false
fiscal_year	FY2024

Pre-populated for all dates in the analysis range (e.g. 2000–2030).

Slowly Changing Dimensions (SCD)

The Problem

Dimension data **changes over time** (e.g. a customer moves cities). How do we handle this in a non-volatile DWH?

SCD Type 1 – Overwrite

Simply update the old value. No history is kept.

Use when history does not matter.

SCD Type 2 – Add a New Row

Insert a new row with the new value and a validity date range.

Old rows are kept with their original values.

Most common approach.

SCD Type 3 – Add a New Column

Add a column for “previous value”.

Only works for one level of history.

Definition

A **data mart** is a subset of the Data Warehouse focused on a **specific business domain** or department.

- Examples: Sales Mart, Finance Mart, HR Mart
- Contains only the data relevant to that domain
- Faster queries due to smaller scope
- Can be sourced from the central DWH (*dependent data mart*) or built independently (*independent data mart*)

Independent Data Marts: A Warning

Multiple independent data marts often lead to **data silos**: inconsistent definitions, duplicated data, conflicting numbers. This is sometimes called a “*data mart sprawl*”.

Table of Contents

- 1 Introduction
- 2 Data Warehouse
- 3 DWH Architectures**
- 4 Creating a Data Warehouse
- 5 OLAP & Business Intelligence

Overview: DWH Architecture Layers

A modern Data Warehouse is built in **logical layers**, each with a specific responsibility.

Picture: DWH Layer Architecture

Add a vertical stack diagram showing: Source Systems → Staging Area → Core DWH (ODS / Integration Layer) → Data Marts → BI & Reporting Tools. Use arrows to show data flow direction.

What lives here?

- All **operational systems** that produce data
- Can be heterogeneous: relational DBs, ERP, CRM, flat files, APIs, IoT streams, spreadsheets
- Data is typically in OLTP-normalised form

Challenges

- Different data formats, encodings, and naming conventions
- Data quality issues: missing values, duplicates, inconsistencies
- Access restrictions – not all source systems expose data easily
- Systems may not be under DWH team's control

Staging Area

Purpose

A **temporary landing zone** where raw data from source systems is loaded before any transformation.

- Data is copied **as-is** from the sources (no transformation yet)
- Allows validation and profiling before touching the core DWH
- Decouples source systems from the integration layer
- Usually **truncated and reloaded** with each ETL run
- Not typically accessible to end users

Examples

Analogy Think of the staging area as a **receiving dock** in a warehouse. Goods arrive, are checked, and only then moved to the shelves.

Purpose

The **single source of truth**: cleaned, integrated, historically complete data for the entire enterprise.

- Data is transformed, validated, and harmonised
- Stores **full history** of changes
- Schema is typically highly normalised (3NF) in Inmon's approach or uses dimensional modelling (Kimball's approach)
- This layer feeds the data marts

Two Common Modelling Approaches

Inmon (Top-Down)

Central, normalised DWH first
Consistent & governed
Higher upfront cost

Kimball (Bottom-Up)

Build data marts first
Faster time-to-value
Risk of mart sprawl

Inmon vs. Kimball Architecture

Picture: Inmon vs. Kimball Comparison Diagram

Add a side-by-side diagram: Left side (Inmon): Sources → ETL → Centralised Normalised DWH → Dependent Data Marts. Right side (Kimball): Sources → ETL → Multiple Star-Schema Data Marts → Bus Architecture / Conformed Dimensions.

Purpose

Subject-specific views of the DWH data, optimised for **end-user queries** in a particular business domain.

- Star or snowflake schema (dimensional model)
- Typically read by BI tools, dashboards, and analysts
- Aggregated or pre-computed data speeds up reporting

Examples

Examples

- **Sales Mart:** revenue, orders, customer acquisition
- **Finance Mart:** P&L, cost centres, budget vs. actuals
- **HR Mart:** headcount, turnover, absenteeism

ETL: Extract, Transform, Load

The data pipeline between layers

- **Extract:** Read data from source systems (full or delta)
- **Transform:** Cleanse, harmonise, aggregate, enrich
- **Load:** Write the result into the DWH target tables

Picture: ETL Pipeline Diagram

Add a horizontal flow diagram: Source DB / File / API → [Extract] → Staging → [Transform & Cleanse] → Integration Layer → [Load] → Data Mart. Annotate with example transformations (date normalisation, deduplication, ...).

ETL vs. ELT

Traditional ETL

Transform data **before** loading it into the DWH.

Transformation logic runs in a dedicated ETL tool or server.

Modern ELT (Extract, Load, Transform)

Load raw data **first**, then transform it *inside* the DWH.

Made possible by powerful cloud DWH engines (Snowflake, BigQuery, Redshift).

	ETL	ELT
Transform where?	External tool	Inside DWH
Data in DWH	Clean only	Raw + clean
Flexibility	Lower	Higher
Typical context	On-premise	Cloud

The Operational Data Store (ODS)

What is an ODS?

An intermediate database that integrates data from multiple sources in **near real-time**, before it reaches the DWH.

- Contains **current** (or very recent) integrated data
- Can be queried for operational reporting (not historical analysis)
- Serves as a buffer between OLTP sources and the DWH
- Supports short-term decision making, while the DWH handles strategic analytics

ODS vs. DWH

ODS

Current / recent data
Updated frequently
Operational use

DWH

Full history
Loaded in batches
Analytical / strategic use

Lambda Architecture (Modern DWH)

Concept

Combines **batch processing** (historical, high-throughput) with **stream processing** (real-time, low-latency) in a unified architecture.

- **Batch layer:** processes all historical data periodically
- **Speed layer:** processes new events in real time
- **Serving layer:** merges results from both for queries

Why it matters

Many businesses need **both**: historical trend analysis *and* real-time dashboards (e.g. fraud detection).

Note (!)

Lambda Architecture is an advanced/supplementary topic. It will not appear in the exam.

Data Lake vs. Data Warehouse

Property	Data Warehouse	Data Lake
Data format	Structured	Any (raw)
Schema	Schema-on-write	Schema-on-read
Data quality	High (cleansed)	Variable (raw)
Users	Analysts, BI tools	Data scientists, ML
Query engine	SQL	SQL, Python, Spark ...
Typical tech	Snowflake, BigQuery	S3, ADLS, Hadoop
Cost	Higher (processing)	Lower (storage)

Data Lakehouse (!)

A newer paradigm that combines the flexibility of a data lake with the governance and performance of a DWH (e.g. Databricks Delta Lake).

Table of Contents

- 1 Introduction
- 2 Data Warehouse
- 3 DWH Architectures
- 4 Creating a Data Warehouse**
- 5 OLAP & Business Intelligence

Overview: The DWH Development Process

Building a DWH is an iterative, multi-phase project.

Picture: DWH Development Lifecycle

Add a circular/iterative lifecycle diagram with phases: (1) Requirements Analysis → (2) Source Analysis → (3) Data Modelling → (4) ETL Design & Dev → (5) Testing → (6) Deployment → (7) Monitoring & Maintenance → back to (1).

Key Principle

Do **not** try to build everything at once. Start with a focused scope (one subject area / one data mart) and expand iteratively.

Phase 1: Requirements Analysis

Goal

Understand **what questions** the business needs to answer.

- Interview stakeholders: managers, analysts, power users
- Identify **Key Performance Indicators** (KPIs)
- Determine the required **granularity** of data
- Define the **history depth** needed (1 year? 10 years?)
- Understand refresh frequency: daily batch? Near real-time?

Examples

Questions to ask stakeholders

- *“What decisions does your department make weekly?”*
- *“What data do you currently look up manually?”*
- *“What would you do if you had this data instantly?”*

Phase 2: Source System Analysis

Goal

Understand and document the **data landscape**: what data exists, where it lives, and how good it is.

- Inventory all source systems and their owners
- Understand the data model of each source (schema, relationships)
- Assess **data quality**: completeness, accuracy, consistency
- Identify **change detection**: how to know what changed since last load? (timestamps, CDC, log-based, full-diff)
- Negotiate **data access**: APIs, DB connections, file exports

Common Surprise

Data quality in source systems is **almost always worse** than stakeholders expect. Plan for data cleansing effort.

Kimball's Dimensional Modelling Steps

Ralph Kimball defined a 4-step process for designing the dimensional model:

- ❶ **Select the business process**
e.g. “order management”, “customer payments”
- ❷ **Declare the grain**
The most atomic level of data to store.
e.g. “one row per order line item”
- ❸ **Identify the dimensions**
Who, what, when, where, why – the context of each fact.
- ❹ **Identify the facts**
The numeric measurements: revenue, quantity, duration, ...

Grain First!

Declaring the grain **before** choosing facts or dimensions is the most important step. Everything else follows from it.

Phase 3: Data Modelling – Star Schema Design

Designing the Fact Table

- One row = one **event at the declared grain**
- Contains foreign keys to all dimension tables
- Contains the **additive measures** (facts): amount, count, duration
- Avoid storing textual descriptions in the fact table

Additive vs. Non-Additive Facts

Type	Description
Additive	Can SUM across all dimensions (e.g. revenue)
Semi-additive	Can SUM across some dimensions (e.g. balance)
Non-additive	Cannot SUM meaningfully (e.g. ratio, price)

Phase 4: ETL Development

Extract

- Connect to source system and pull data (full or incremental)
- **Change Data Capture (CDC)**: only extract what changed
- Common CDC methods: timestamp columns, database triggers, log mining

Transform

- Cleanse: handle nulls, remove duplicates, fix encodings
- Harmonise: map codes to uniform values (e.g. “M” and “male” → “Male”)
- Derive new attributes (e.g. age from birth date)
- Apply **surrogate keys**: replace natural keys with integer DWH keys

Load

- Insert new dimension records; apply SCD logic for changes
- Insert new fact rows (usually append-only)

Natural Key vs. Surrogate Key

- **Natural key:** the business identifier from the source system (e.g. customer number, product code)
- **Surrogate key:** a meaningless integer generated by the DWH (e.g. 1, 2, 3, ...)

Why Use Surrogate Keys?

- Natural keys can change in source systems
- Surrogate keys remain stable – essential for SCD Type 2
- Smaller keys = faster joins (integer vs. string)
- Decouples the DWH from source system changes

Data Quality in the ETL Process

Common Data Quality Issues

- **Completeness:** missing or NULL values
- **Consistency:** same entity represented differently (“GmbH” vs. “Gesellschaft mbH”)
- **Accuracy:** values that are wrong or outdated
- **Timeliness:** data that arrives too late to be useful
- **Duplicates:** same record loaded multiple times

Strategies

- Validate at load time; reject or quarantine bad records
- Use a **“bad data” / error table** for rejected rows
- Implement data quality metrics and monitoring dashboards
- Work with source system owners to fix issues at the source

Phase 5–7: Testing, Deployment, and Monitoring

Testing

- **Row count checks:** did all expected rows load?
- **Reconciliation:** totals in DWH match totals in source
- **Regression tests:** check that historical data has not changed
- **Business validation:** have domain experts sign off on reports

Deployment & Monitoring

- Automate ETL jobs with a scheduler (e.g. Apache Airflow, cron)
- Monitor job run times and failure rates
- Alert on late arrivals, row count deviations, or quality failures
- Maintain **lineage documentation:** where does each column come from?

Common DWH Tools (Overview)

ETL / ELT Tools

- Traditional: **Informatica PowerCenter**, **SSIS** (Microsoft), **Talend**
- Modern / cloud-native: **dbt** (data build tool), **Apache Spark**, **Fivetran**

DWH Storage Engines

- On-premise: **Teradata**, **SAP BW**, **Oracle DW**
- Cloud: **Snowflake**, **Google BigQuery**, **Amazon Redshift**, **Azure Synapse**

Note (!)

Tool knowledge is supplementary. The concepts and architecture are what matter for this course.

Table of Contents

- 1 Introduction
- 2 Data Warehouse
- 3 DWH Architectures
- 4 Creating a Data Warehouse
- 5 OLAP & Business Intelligence**

What is OLAP?

Online Analytical Processing (OLAP)

A category of software technology that enables analysts to **interactively and rapidly analyse multidimensional data** from multiple perspectives.

- Coined by Edgar F. Codd (1993) – same person as the relational model
- Data is conceptually stored as a **multidimensional cube**
- Analysts “slice” the cube to get the view they need
- Queries are typically aggregations: SUM, AVG, COUNT, MIN, MAX

Examples

Example Questions OLAP Answers

- *What were total sales per region per quarter in 2023?*
- *Which product category grew fastest in the last 3 years?*
- *Compare Germany vs. France revenue for Q2 this year.*

The OLAP Cube

Concept

Data is organised as a **multidimensional hypercube**: each dimension forms one axis, each cell contains the aggregated measure.

Picture: 3D OLAP Cube

Add a 3D cube diagram with three axes: $X = \text{Time (Q1, Q2, Q3, Q4)}$, $Y = \text{Product Category (Electronics, Clothing, Food)}$, $Z = \text{Region (North, South, East, West)}$. Highlight one cell to show a single aggregate value (e.g. "Revenue = €42,500").

- Real cubes can have **more than 3 dimensions**
- Cells that have no data are called **sparse** (common in practice)

OLAP Operations: Roll-Up and Drill-Down

Roll-Up (Aggregation)

Move **up** the dimension hierarchy – *less detail, more aggregation*.

Example: Month → Quarter → Year

Drill-Down (Disaggregation)

Move **down** the dimension hierarchy – *more detail, less aggregation*.

Example: Year → Quarter → Month → Day

Picture: Roll-Up / Drill-Down Hierarchy

Add a tree diagram for the Date dimension: All Time → Year → Quarter → Month → Day. Use bidirectional arrows with “Roll-Up” and “Drill-Down” labels.

OLAP Operations: Slice, Dice, and Pivot

Slice

Select a **single value** on one dimension, reducing the cube by one dimension.
Example: “Show me only Q1 2024” – fixes the Time axis.

Dice

Apply **multiple filters** across multiple dimensions, producing a sub-cube.
Example: “Q1 and Q2, Electronics and Clothing, Germany only.”

Pivot (Rotate)

Re-orient the cube by swapping row and column dimensions.
Example: Rows = Products, Columns = Regions $\Leftrightarrow$ Rows = Regions, Columns = Products.

Dimension Hierarchies

Definition

A **hierarchy** is a structured set of levels within a dimension, from coarse to fine granularity.

Common Hierarchies

Dimension	Hierarchy
Time	Year → Quarter → Month → Week → Day
Location	Continent → Country → Region → City
Product	Category → Subcategory → Product → SKU
Organisation	Company → Division → Department → Employee

Hierarchies enable **roll-up** and **drill-down** navigation.

OLAP Variants: ROLAP, MOLAP, HOLAP

ROLAP – Relational OLAP

Data stored in a **relational DWH** (star/snowflake schema). Cube operations translated to SQL at query time.

Scalable; no pre-computation needed; slower for complex aggregations.

MOLAP – Multidimensional OLAP

Data stored in a **proprietary multidimensional cube** (pre-aggregated).

Very fast queries; limited scalability; storage overhead.

HOLAP – Hybrid OLAP

Detailed data in relational storage; aggregations in multidimensional cube.

Balances scalability and performance.

SQL Extensions for OLAP

Standard SQL has been extended with OLAP-specific syntax.

GROUP BY ROLLUP

Generates subtotals and a grand total across a hierarchy.

```
SELECT region, product, SUM(revenue)
FROM fact_sales
GROUP BY ROLLUP(region, product);
```

GROUP BY CUBE

Generates *all possible* grouping combinations.

```
SELECT region, product, year, SUM(revenue)
FROM fact_sales
GROUP BY CUBE(region, product, year);
```


SQL Window Functions for Analytics

OVER Clause

Window functions perform calculations **across a set of rows** related to the current row, without collapsing them.

```
-- Running total of revenue per region
SELECT
    date_key,
    region,
    revenue,
    SUM(revenue) OVER (
        PARTITION BY region
        ORDER BY date_key
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_total
FROM fact_sales;
```

Common Window Functions

ROW_NUMBER(), RANK(), DENSE_RANK(), LAG(), LEAD(), NTILE(),
PERCENT_RANK()

GROUPING SETS

Allows specifying **exactly which grouping combinations** to compute in a single query – more flexible than ROLLUP or CUBE.

```
SELECT region, product, year, SUM(revenue)
FROM fact_sales
GROUP BY GROUPING SETS (
    (region, year),      -- revenue by region & year
    (product, year),     -- revenue by product & year
    (region),            -- revenue by region only
    ()                   -- grand total
);
```

Key Benefit

One query replaces multiple UNION ALL subqueries. The database can optimise the single pass over the data.

What is Business Intelligence?

Definition

Business Intelligence (BI) is a set of technologies, processes, and practices that transform raw data into **meaningful insights** to support better business decisions.

- BI sits **on top of** the Data Warehouse and OLAP layer
- Includes: reporting, dashboards, data visualisation, ad-hoc queries
- Target users: executives, managers, business analysts (non-technical)
- Goal: enable **data-driven decision making** across the organisation

Picture: The Full BI Stack

Add a layered stack diagram: (Bottom) Source Systems → ETL / ELT → Data Warehouse / Data Lake → OLAP / Semantic Layer → (Top) BI Tools / Dashboards / Reports. Add example product names at each layer (SAP, Airflow, Snowflake, SSAS, Power BI / Tableau).

Types of BI Analysis

Descriptive Analytics – *What happened?*

Summarises historical data. Standard reports and dashboards.

Example: Monthly revenue report.

Diagnostic Analytics – *Why did it happen?*

Drill-down analysis to find root causes.

Example: Revenue dropped in Q3 – which region? Which product?

Predictive Analytics – *What will happen?*

Uses statistical models and ML to forecast future outcomes.

Example: Demand forecasting, churn prediction.

Prescriptive Analytics – *What should we do?*

Recommends actions based on data and models.

Example: Automated pricing engine, resource scheduling.

KPIs and Metrics

Key Performance Indicator (KPI)

A quantifiable measure used to evaluate success in achieving objectives.

Properties of a Good KPI

- **Specific:** tied to a concrete business goal
- **Measurable:** expressed as a number
- **Actionable:** something the business can actually influence
- **Timely:** available when decisions need to be made

Examples

Example KPIs

- Monthly Recurring Revenue (MRR)
- Customer Acquisition Cost (CAC)
- Net Promoter Score (NPS)
- Inventory Turnover Rate

Dashboards and Reporting

Operational Reports

Fixed-format, scheduled reports delivered to users regularly.

Example: End-of-day sales summary sent every evening.

Ad-Hoc Analysis

Users create their own queries and reports on demand, without IT involvement.

Requires a user-friendly semantic layer.

Interactive Dashboards

Visual, real-time displays of KPIs. Users can filter, drill down, and explore data interactively.

Tools: **Power BI**, **Tableau**, **Looker**, **Metabase**

Picture: Example Dashboard Mockup

Add a mockup or screenshot of a typical BI dashboard with: bar chart (revenue by region), line chart (revenue trend), KPI tiles, and a data table. Can use a Power BI or Tableau screenshot.

Definition

A **semantic layer** (also called a “business layer”) maps the physical DWH tables and columns to **business-friendly terms** that non-technical users understand.

- Abstracts SQL complexity from business users
- Defines metrics, KPIs, and dimensions in business language
- Ensures consistent definitions company-wide (“Revenue” always means the same thing)
- Examples: **SSAS Tabular, Looker LookML, dbt metrics, Power BI datasets**

Summary: From Data to Decision

Picture: End-to-End Data Flow

Add a horizontal arrow diagram showing the complete journey: Raw Transactions (OLTP) → ETL/ELT → Data Warehouse → OLAP Cube / Semantic Layer → BI Dashboard → Business Decision. Add example artefacts at each step (invoice, DIM_CUSTOMER table, cube, Power BI report, board decision).

Key Takeaways

- OLTP systems **generate** data; DWH systems **integrate** it
- OLAP enables **multidimensional analysis** via cubes and hierarchies
- BI tools **present** insights to decision makers
- Data quality throughout the pipeline determines the value of analytics